

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Towards Formalizing Higher-Order Pattern Unification in Isabelle/HOL

Bachelor Thesis

Roman Angehrn

September 3, 2020

Advisors: Dr. Dmitriy Traytel, Joshua Schneider
Professor: Prof. Dr. David Basin

Department of Computer Science, ETH Zürich

Abstract

In this thesis, we work towards formally proving the correctness of Nipkow's algorithm for higher-order pattern unification. The formalization is carried out in the Isabelle/HOL proof assistant. We closely follow the pen-and-paper proofs by Grobauer and add many missing details. We discuss how we extended some of the proofs to make them work in Isabelle. In particular, we show that some of Grobauer's results do not hold for untyped terms. We also show how this extends to the unification algorithm. The Isabelle formalization includes proofs of most of the auxiliary lemmas used by Grobauer and a first step towards proving termination of the unification algorithm. Proving the correctness of the unification algorithm remains to be done.

Contents

Contents **iii**

1 Introduction **1**

2 Preliminaries **3**
2.1 Lambda Calculus 3
2.1.1 β- and η-Reduction 4
2.1.2 Substitution for Free Variables 6
2.2 Unification 6

3 Types and Most General Unifiers **9**

4 Formalization of Lambda Terms **13**
4.1 Representation of Lambda Terms 13
4.2 Formalizing the Lambda Calculus 14
4.2.1 Lemmas from Grobauer's Thesis 15
4.2.2 General Facts about the Lambda Calculus 18

5 The Unification Algorithm and Formal Proof of Correctness **19**

6 Conclusion **23**

Bibliography **25**

Chapter 1

Introduction

Unification is the process of trying to make two expressions of some language equal by instantiating their variables appropriately. Depending on the language of the expressions there might be multiple solutions and a special solution, the most general unifier, which "includes" all other possible solutions (see Section 2.2 for more detail). On the other hand, there might not be a solution at all if the two terms are not unifiable.

Unification is used in a variety of applications. One example is type inference where unification can be used to find a solution to a set of constraints that were gathered to determine the most general type of some expression. Another example are proof assistants where unification can be used to determine if a rule can be applied to a set of terms. In a similar way, unification can be used to find out if a rule of a rewrite system matches a term.

In this thesis, we work towards formally proving the correctness of Nipkow's unification algorithm for higher-order lambda patterns [5] in Isabelle/HOL. We use Grobauer's more formal development of the same algorithm [1] as a basis. We formalize the proofs of most of the auxiliary lemmas used by Grobauer. We discover that for a few of these lemmas the formal proof requires significantly more work and even additional assumptions which are not mentioned explicitly in Grobauer's thesis. Particularly, we find that one of the lemmas requires the lambda terms in question to be simply typed, and we present an untyped lambda term which serves as a counterexample. This extends to the unification algorithm which will not necessarily find the most general unifier if we consider untyped terms. We also discuss how the unification algorithm can be implemented in Isabelle.

Constructing a formal proof in a proof assistant such as Isabelle has the advantage that we can be quite certain that the proof actually works. The proof assistant will check that every step of the proof is justified by logical axioms or by previously proved rules. If we trust the proof assistant and we

trust that the formalization expresses what we wanted to express, then we can be certain that the statement we proved is correct. Additionally, using a proof assistant can help with finding mistakes in the argumentation of a pen-and-paper proof. On the other hand, constructing proofs in a proof assistant can be cumbersome because every concept has to be precisely defined, and even though the proof assistant will apply simple proof steps on its own, formal proofs often have to be very elaborate.

We encountered both the advantages and the disadvantages of formalization in this thesis. Translating existing proofs to Isabelle was sometimes very time consuming, but as mentioned above, we were able to find some issues in the pen-and-paper proofs we formalized.

Unfortunately, due to time constraints, it was not possible to formally prove the correctness of the unification algorithm, so this thesis can be seen as a starting point towards a formal proof of correctness or as an extension of the existing implementation of the lambda calculus in the Isabelle/HOL distribution.

Chapter 2 introduces the concepts and notation necessary to understand the rest of the thesis. Chapter 3 explains why types are necessary to obtain most general unifiers. Chapter 4 describes the formalization of the lambda calculus, based on the existing implementation in the Isabelle/HOL distribution and on Grobauer's thesis [1]. Chapter 5 presents the unification algorithm and its implementation in Isabelle and the first steps towards proving termination. Finally, Chapter 6 summarizes the thesis and provides an outlook.

Related Work The unification algorithm discussed in this thesis was proposed by Nipkow [5]. Grobauer [1] presented a more formal development of the same algorithm which we use as a basis. Huet [2] showed that general higher-order unification is undecidable and presented a semi-decision algorithm [3]. Miller [4] discovered higher-order patterns and showed that unification of higher-order patterns is a decidable subset of general higher-order unification.

Chapter 2

Preliminaries

This chapter introduces the concepts and the notation that will be used throughout the thesis. The notation and the definitions are very similar to the ones used by Grobauer [1].

2.1 Lambda Calculus

Throughout the thesis, we use the lambda calculus in either its untyped or simply typed variant. Where there is a difference between the two, it will be mentioned which one is used (cf. Chapter 3). We use the following syntax:

$$t ::= x \mid F \mid \lambda\ t \mid t\,@\,t$$

where @ denotes function application, x a bound variable and F a free variable. There are two notable changes to the usual lambda notation which are explained in the following.

We syntactically distinguish between free and bound variables. This is useful when dealing with unification, as it allows us to easily determine if a variable is free or bound, even if we are looking at a subterm occurring under an abstraction. This does however, come with the drawback that the meaning of bound variables is not as clear anymore. In particular, we can now have terms where there is a bound variable with no abstraction binding it. We call such an occurrence of a bound variable a loose bound variable. We will usually prevent this situation by requiring terms not to contain any loose bound variables, however if we are looking at a subterm (which we will do as part of the unification algorithm), this case will still occur.

We use de Bruijn indices to denote bound variables. See Section 4.1 for a discussion of advantages and disadvantages of de Bruijn indices compared to named bound variables. We denote an occurrence of a bound variable

with i abstractions between it and the abstraction where it is bound by $\underline{i}$. In the formalization, free variables will also be denoted by natural numbers, but in their case the number is just a name for that variable and does not have the same meaning as for bound variables. Moreover there can be a bound variable x and a free variable F with the same natural number for the index. For a loose bound variable, the meaning of its index as described above no longer makes sense, as there is no abstraction binding it. In that case, the index just serves as a name.

Example 2.1 *In the term $\lambda\ \underline{0} \mathbin{@} \underline{1}$ the bound variable occurring on the right side of the application is a loose bound variable.*

We will use the following letters to denote (parts of) lambda terms. The letters r, s and t denote arbitrary lambda terms as defined above. The letters p, q, x, y and z denote bound variables, $x + i$ for some natural number i denotes the bound variable we obtain by increasing the index of x by i. The letters D, E, F, G and H denote free variables, and the letters f, g and h denote either bound or free variables.

We will denote a list of arbitrary lambda terms by $\overline{t_n}$, where n denotes the length of the list. Similarly, a list of bound variables of length n will be denoted by $\overline{x_n}$. The indices to the individual elements of these lists appear in decreasing order, starting at $n-1$ and going down to 0: $\overline{t_n} = [t_{n-1}, \ldots, t_0]$. Lists with unspecified lengths will be denoted by ts and xs respectively and their lengths are $|ts|$ and $|xs|$. We use the following notation for the standard list functions *cons* and *append*: $x_m{:}[x_{m-1}, \ldots, x_0] = [x_m, x_{m-1}, \ldots, x_0]$ and $[x_m, \ldots, x_0] +\!\!+ [y_n, \ldots, y_0] = [x_m, \ldots, x_0, y_n, \ldots, y_0]$. The set of elements of a list is denoted by $\{\overline{x_n}\}$. A function t applied iteratively to a list of arguments $(\ldots(t \mathbin{@} s_{n-1}) \cdots \mathbin{@} s_0)$ will be abbreviated as $t(\!|\overline{s_n}|\!)$. m nested abstractions $\lambda\ \ldots \lambda\ t$ will be abbreviated as $\lambda_m\ t$

2.1.1 β- and η-Reduction

β- and η-reduction are defined in the usual way, however because we are using de Bruijn indices, the definitions are somewhat more complex.

β-reduction is defined by the following rules:

$$\frac{}{(\lambda\ s) \mathbin{@} t \longrightarrow_\beta s\,[t/0]} \quad \frac{s \longrightarrow_\beta t}{r \mathbin{@} s \longrightarrow_\beta r \mathbin{@} t} \quad \frac{s \longrightarrow_\beta t}{s \mathbin{@} r \longrightarrow_\beta t \mathbin{@} r} \quad \frac{s \longrightarrow_\beta t}{\lambda\ s \longrightarrow_\beta \lambda\ t} \tag{2.1}$$

where $s\,[t/i]$ denotes the substitution of t for all occurrences of the bound variable $\underline{i}$ in s. The definition of this substitution function, which we will

refer to as *subst*, is given in the following.

$$
\begin{aligned}
&t\,[s/k] = \textbf{case}\ t\ \textbf{of}\\
&\quad \mid \underline{i} \to \textbf{if}\ k < i\ \textbf{then}\ \underline{i-1}\ \textbf{else if}\ k = i\ \textbf{then}\ s\ \textbf{else}\ \underline{i}\\
&\quad \mid r\,@\,r' \to (r\,[s/k])\,@\,\left(r'\,[s/k]\right)\\
&\quad \mid \lambda\ r \to \lambda\ \ (t\,[\text{lift}\ s\ 0/k+1])\\
&\quad \mid \textbf{otherwise} \to t
\end{aligned}
$$

$$
\begin{aligned}
&\text{lift}\ t\ k = \textbf{case}\ t\ \textbf{of}\\
&\quad \mid \underline{i} \to \textbf{if}\ i < k\ \textbf{then}\ \underline{i}\ \textbf{else}\ \underline{i+1}\\
&\quad \mid r\,@\,r' \to (\text{lift}\ r\ k)\,@\,(\text{lift}\ s\ k)\\
&\quad \mid \lambda\ s \to \lambda\ \ (\text{lift}\ s\ (k+1))\\
&\quad \mid \textbf{otherwise} \to t
\end{aligned}
$$

Because we are using de Bruijn notation, some shifting of the indices for the bound variables is required to keep them consistent across the substitution. Specifically, we need to decrease all bound variables with indices greater than i by one because the β-reduction removes an abstraction between the occurrence of the bound variable and the abstraction binding it (this is also done for loose bound variables because the abstraction binding the variable might be outside of the subterm we are currently looking at). In the definition above, this is done in the case where s is a bound variable. Moreover, we need to increase i and all loose bound variables in t by one if we encounter an abstraction and proceed with substituting under the abstraction. This is to avoid capturing existing variables and is achieved with the help of the *lift* function.

η-reduction is defined by the following rules:

$$
\frac{\neg\,(\text{free}\ s\ 0)}{\lambda\ (s\,@\,\underline{0}) \longrightarrow_\eta \text{decr}\ s\ 0}\quad
\frac{s \longrightarrow_\eta t}{r\,@\,s \longrightarrow_\eta r\,@\,t}\quad
\frac{s \longrightarrow_\eta t}{s\,@\,r \longrightarrow_\eta t\,@\,r}\quad
\frac{s \longrightarrow_\eta t}{\lambda\ s \longrightarrow_\eta \lambda\ t}
\tag{2.2}
$$

Similar to β-reduction, some shifting of the indices is necessary. In the case of η-reduction, we need to decrease all bound variables by one because we remove an abstraction. This is done by the *decr* function, which is just an application of the *subst* function.

$$\text{decr}\ t\ i = t\,[\underline{i}/i]$$

The function *free* determines if a variable occurs as a loose bound variable in a term and is defined as follows.

$$
\begin{aligned}
&\text{free } t\ i = \textbf{case } t \textbf{ of}\\
&\quad \mid \underline{j} \to (i = j)\\
&\quad \mid r\,@\,s \to (\text{free } r\ i) \vee (\text{free } s\ i)\\
&\quad \mid \lambda\ s \to \lambda\ \ (\text{free } s\ (i+1))\\
&\quad \mid \textbf{otherwise} \to \text{false}
\end{aligned}
$$

We denote the reflexive, transitive and symmetric closure of $\beta \cup \eta$ with $\equiv$ and call two terms satisfying this relation $\beta\eta$-equivalent. We also use the notation $\longrightarrow_{\beta\eta}$ instead of $\beta \cup \eta$.

2.1.2 Substitution for Free Variables

In addition to the *subst* function used for β-reduction which replaces bound variables, we define another substitution function *mk_subst*, which replaces free variables. As there is no shifting to be done, the definition is very simple.

$$
\begin{aligned}
&\text{mk_subst } \xi\ t = \textbf{case } t \textbf{ of}\\
&\quad \mid F \to (\xi F)\\
&\quad \mid x \to x\\
&\quad \mid \lambda\ t' \to \lambda\ (\text{mk_subst } \xi\ t')\\
&\quad \mid r\,@\,s \to (\text{mk_subst } \xi\ r)\,@\,(\text{mk_subst } \xi\ s)
\end{aligned}
$$

In the above definition ξ represents a function mapping free variables to lambda terms. We require ξ to only map to lambda terms without loose bound variables.

We denote the empty substitution by $\emptyset$. Moreover, we denote the substitution replacing F by t and leaving all other variables unchanged by $\{F \mapsto t\}$. We call such a substitution a binding. We define the composition of substitutions as the usual function composition and denote it by $\circ$.

Similar to the *free* function, which determines if a bound variables occurs in a term, we denote the set of free variables occurring in a term t by $\mathcal{FV}(t)$.

2.2 Unification

Unification is the problem of equalizing two terms by finding values to substitute for their respective free variables. A possible outcome is finding that they cannot be equalized in such a way. In our case, we are interested in unifying terms of the lambda calculus. We call the pair of input terms s and

t that we want to equalize (unify) a unification problem and denote it by "$s \stackrel{?}{=} t$". A solution (unifier) to a unification problem is given by a substitution that replaces the free variables occurring in the unification problem such that the two terms will be $\beta\eta$-equivalent. In most cases where the unification problem is solvable (unifiable), there are multiple unifiers. In such a case, we are interested in finding the so called most general unifier (mgu). This is a unifier with the property that any other unifier can be obtained by composing the mgu with another substitution. That is, φ is an mgu if for every unifier σ, there exists a substitution ξ such that $\sigma \equiv \xi \circ \varphi$ where we slightly abuse the $\equiv$ notation to mean $\sigma t \equiv (\xi \circ \varphi)\, t$ for any term t. This abuse of notation will be used throughout the thesis. Obviously, if there is only a single unifier, it is also the mgu.

Example 2.2 *$x \stackrel{?}{=} y$ for two (loose) bound variables x and y is unifiable if and only if $x = y$, in which case the unique unifier is the empty substitution $\emptyset$.*

Example 2.3 *$F \stackrel{?}{=} x$ is unifiable with the unique unifier $\{F \mapsto x\}$.*

Example 2.4 *$F \stackrel{?}{=} G$ is unifiable and the most general unifier is $\{F \mapsto H\} \circ \{G \mapsto H\}$. H is a new free variable and can be instantiated arbitrarily.*

The above examples are all cases of first-order unification which only considers input terms where the free variables that occur are not applied to any arguments. In contrast, higher-order unification allows all lambda terms as inputs and is therefore more general than first-order unification. Higher-order pattern unification is in between these two, allowing only higher-order patterns as input terms. For higher-order patterns, we use the following definition by Nipkow [5].

Definition 2.5 *A term t in β-normal form is a higher-order pattern if every free occurrence of a variable F is in a subterm $F(\!|rrs|\!)$ of t such that rrs is η-equivalent to a list of distinct bound variables.*

Example 2.6 *F, $F \mathbin{@} x$ and $F \mathbin{@} (\lambda\, x \mathbin{@} \underline{0})$ are patterns. $F(\!|\overline{x_n}|\!)$ is a pattern if $x_{n-1}, \ldots, x_0$ are distinct. $F \mathbin{@} G$ is not a pattern.*

General higher-order unification is undecidable as shown by Huet [2]. An algorithm that always finds a unifier if one exists, but may not terminate if there is no unifier was presented by Huet [3]. Higher-order pattern unification is interesting in this sense because it is a decidable subset of general higher-order unification. An algorithm for higher-order unification was presented by Nipkow [5], which is the algorithm we will be looking at in this thesis.

Chapter 3

Types and Most General Unifiers

The unification algorithm presented by Nipkow [5] works for both untyped and simply typed lambda terms. Initially in the development, the lambda terms are assumed to be simply typed and a (sketch of a) proof of correctness is given, showing that the algorithm computes the most general unifier (mgu) if a unifier exists. Later, in one of the refinement steps, the typing assumption is removed. There is, however, no proof of correctness for this version of the algorithm. As we show in the last part of this chapter, when allowing untyped terms, an mgu might not necessarily exist, even if the inputs are unifiable.

In Grobauer's more formal treatment of Nipkow's algorithm [1] it is not mentioned explicitly whether the lambda terms under consideration are assumed to be simply typed or not. The only mention of types is in the abstract where the typed lambda calculus and typed lambda terms are briefly mentioned.

Some of the results from Grobauer's thesis can be shown for untyped lambda terms. In particular, most of the lemmas and facts about the lambda calculus in the appendix and termination of the unification algorithm. The notable exception to this is the following Lemma:

Lemma 22 *Let t have no loose bound variables and let*

$$t(\!|xs +\!\!+ x{:}\overline{x_m}|\!) \equiv t(\!|ys +\!\!+ y{:}\overline{y_m}|\!)$$

for two lists of bound variables $xs +\!\!+ x{:}\overline{x_m}$ and $ys +\!\!+ y{:}\overline{y_m}$. Then

- *if $x = y$ it is $t(\!|xs +\!\!+ z{:}\overline{x_m}|\!) \equiv t(\!|ys +\!\!+ z{:}\overline{y_m}|\!)$ for any bound variable z*
- *if $x \neq y$ there exists a t' such that $t \equiv \lambda_{|xs|+1}\ t'$ and $\neg$ (free t' 0)*

The need for types arises from the following intermediate result in Grobauer's proof of the above Lemma: "Thus there is a t'' with $t\ (\beta \cup \eta)^*\ t''$ such that

$t''(\!|xs +\!\!+ x{:}\overline{x_m}|\!) \xrightarrow{*}_\beta s_1$ and $t''(\!|ys +\!\!+ y{:}\overline{y_m}|\!) \xrightarrow{*}_\beta s_2$ where every β-reduction involves one of the loose bound variables in the argument lists and of course $s_1 = s_2 = s$." ([1] p. 56). After several unsuccessful attempts to formally prove the existence of such a t'', my advisors discovered that the Lemma does not necessarily hold for untyped terms. Specifically, they found the following untyped lambda term which serves as a counterexample:

$$U = (\lambda_3\ ((\underline{2} \mathbin{@} \underline{2}) \mathbin{@} \underline{0}) \mathbin{@} \underline{1}) \mathbin{@} (\lambda_3\ ((\underline{2} \mathbin{@} \underline{2}) \mathbin{@} \underline{0}) \mathbin{@} \underline{1})$$

For clarity, here is the same term with named bound variables:

$$U = (\lambda x.\ \lambda a.\ \lambda b.\ ((x \mathbin{@} x) \mathbin{@} b) \mathbin{@} a) \mathbin{@} (\lambda x.\ \lambda a.\ \lambda b.\ ((x \mathbin{@} x) \mathbin{@} b) \mathbin{@} a)$$

U cannot be typed because it contains terms which are applied to themselves.

We obtain the counterexample by choosing $t = U$, $xs = ys = []$, $\overline{x_m} = [y]$ and $\overline{y_m} = [x]$ such that $x \neq y$. Obviously, U has no loose bound variables. Furthermore, we have

$$U \longrightarrow_\beta \lambda_2\ (U \mathbin{@} \underline{0}) \mathbin{@} \underline{1}$$

and therefore

$$U(\!|x, y|\!) \xrightarrow{*}_\beta U(\!|y, x|\!)$$

so $U(\!|x, y|\!) \equiv U(\!|y, x|\!)$. Lemma 22 then states that there exists a t' such that $U \equiv \lambda\ t'$ and $\neg\,(\text{free}\ t'\ 0)$.

Because of the Church-Rosser property of $\beta \cup \eta$, we know that there exists a t'' such that $U \xrightarrow{*}_{\beta\eta} t''$ and $\lambda\ t' \xrightarrow{*}_{\beta\eta} t''$. We now look at the possible reductions in the chain $\lambda\ t' \xrightarrow{*}_{\beta\eta} t''$. Since $\neg\,(\text{free}\ t'\ 0)$, the first reduction has to be of the form $\lambda\ t' \longrightarrow_{\beta\eta} \lambda\ t_3$ for some term t_3 where $t' \longrightarrow_{\beta\eta} t_3$. Moreover, because β- and η-reductions do not introduce new loose bound variables, we know that $\neg\,(\text{free}\ t_3\ 0)$. Using the same argument, we can conclude that all following reductions are of the same form and do not remove the abstraction. Therefore t'' is of the form $\lambda\ t_4$ for some t_4, where $t' \xrightarrow{*}_{\beta\eta} t_4$ and $\neg\,(\text{free}\ t_4\ 0)$.

Making use of the fact that β- and η-reductions do not introduce new loose bound variables again and using $U \xrightarrow{*}_{\beta\eta} t''$, we can conclude that t'' does not contain any loose bound variables. Together with $t'' = \lambda\ t_4$, this implies that t_4 has no loose bound variables with index greater than zero. Combined with $\neg\,(\text{free}\ t_4\ 0)$, we find that t_4 contains no loose bound variables either.

We now consider the term $U(\!|x, y|\!)$. We know that $U(\!|x, y|\!) \xrightarrow{*}_{\beta\eta} (\lambda\ t_4)\,(\!|x, y|\!)$, moreover, we have $(\lambda\ t_4)\,(\!|x, y|\!) \longrightarrow_\beta t_5 \mathbin{@} y$ where $t_5 = t_4\,[x/0]$. Because $\neg\,(\text{free}\ t_4\ 0)$, the substitution does not substitute x anywhere in t_4 and so we get $t_5 = \text{decr}\ t_4\ 0$. The *decr* function decrements the indices of all loose

bound variables by one. We can therefore conclude that t_5 does not contain any loose bound variables.

We now have a chain of β- and η-reductions, starting from $U(\!|x, y|\!)$ and ending in $t_5 \mathbin{@} y$. Because t_5 does not contain any loose bound variables, we can conclude that $\neg\,(\text{free}\ (t_5 \mathbin{@} y)\ x)$ (we assumed $x \neq y$). Moreover it is obvious that free $U(\!|x, y|\!)\ x$ which implies that x was removed by one of the reductions. This is where we reach a contradiction, as there is no chain of reductions starting from $U(\!|x, y|\!)$ and ending in a term where x does not occur freely, as is shown in the following.

η-reductions do not modify loose bound variables, therefore they cannot remove loose bound variables. Furthermore, the only way for a β-reduction to remove a loose bound variable is if we have a term of the form $(\lambda\ s) \mathbin{@} t$, where $\neg\,(\text{free}\ s\ 0)$. As U does not contain such a term and neither a β- nor an η-reduction can create such a term, it follows that during the chain $U(\!|x, y|\!) \xrightarrow{*}_{\beta\eta} (t_5 \mathbin{@} y)\ x$ is never removed and therefore free $(t_5 \mathbin{@} y)\ x$ in contradiction to the fact that t_5 does not contain loose bound variables.

We can therefore conclude that Lemma 22 does not necessarily hold for untyped terms. Assuming typed terms helps in the formal proof of the Lemma because we can make use of the strong normalization property of the simply typed lambda calculus with respect to β- and η-reductions. This property implies that there are no infinite chains of reduction, i.e. starting at any term and applying any valid β- and η-reductions, we will eventually end up with a term that cannot be reduced any further (normal form).

Lemma 22 is not directly used in the proofs involving the unification algorithm, but it is used to show the following two facts. We can construct a counterexample using U with a similar argument, implying that these facts also do not necessarily hold for untyped terms.

Fact 10 *If t has no loose bound variables and $t(\!|\overline{x_m}|\!) \equiv t(\!|\overline{y_m}|\!)$ for argument lists $\overline{x_m}$ and $\overline{y_m}$ where for $0 \leq i \leq m-1$ it is $x_i \neq y_i$ then there is a t' without loose bound variables such that $t \equiv \lambda_m\ t'$.*

Fact 12 *Let t have no loose bound variables and let $t(\!|\overline{x_m}|\!) \equiv t(\!|\overline{y_m}|\!)$. If for $\overline{z_m}$ and $\overline{z'_m}$ it is $z_i = z'_i$ whenever $x_i = y_i$ then $t(\!|\overline{z_m}|\!) \equiv t(\!|\overline{z'_m}|\!)$.*

In both cases, the counterexample is $t = U$, $\overline{x_m} = \overline{y_m} = [x, y]$ for some bound variables x and y with $x \neq y$. These Lemmas are used in the correctness proof of the unification algorithm. Specifically, they are used for showing the correctness of the *flex-flex-1* case (see Chapter 5).

We can furthermore use the term U to show that Nipkow's algorithm [5] will not necessarily find the mgu when considering untyped terms. In order to do so, we first use the algorithm to solve the unification problem $F \mathbin{@} (\!|x, y|\!) \stackrel{?}{=}$

$F \mathbin{@} (\!|y, x|\!)$ for two bounds variables x and y with $x \neq y$, which produces the unifier $\{F \mapsto \lambda_2\ H\}$. This is the mgu if we only consider simply typed terms. If we allow untyped terms, we can see that $\{F \mapsto U\}$ is a unifier as well. Assuming that $\{F \mapsto \lambda_2\ H\}$ is also the mgu when allowing untyped terms, we should be able to find a substitution φ such that $\varphi \circ \{F \mapsto \lambda_2\ H\} \equiv \{F \mapsto U\}$ (slightly abusing the notation $\equiv$, as described in Section 2.2).

Because U does not contain any free variables it is obvious that φ should be of the form $\{H \mapsto t\}$ for some term t not containing any loose bound variables, cf. Section 2.1.2. The problem of finding a suitable φ is therefore equivalent to the problem of finding a term t that does not contain any loose bound variables and where $\lambda_2\ t \equiv U$. This is a special case of the statement we obtained when instantiating Lemma 22 with our counterexample. We can therefore use the same argument to show that such a term t does not exist. This implies that there is no substitution φ such that $\varphi \circ \{F \mapsto \lambda_2\ H\} \equiv \{F \mapsto U\}$ which further implies that $\{F \mapsto \lambda_2\ H\}$ is not the mgu. We can therefore conclude that Nipkow's algorithm does not necessarily find the mgu when considering untyped terms.

Chapter 4

Formalization of Lambda Terms

4.1 Representation of Lambda Terms

One of the first decisions that we had to make was choosing how to represent lambda terms in Isabelle/HOL, namely whether to use named variables or de Bruijn indices.

Named variables are more natural to understand and easier to use when manipulating lambda terms by hand, especially if the terms contain nested abstractions. On the other hand, implementing lambda terms with named variables in a programming language is not as simple because every term containing variables can be represented in any number of ways by renaming the variables. This issue is mitigated by the fact that the Isabelle/HOL distribution includes a library which provides support for nominal datatypes.

De Bruijn indices are not as natural to use when manipulating terms by hand, but an implementation in a programming language is simpler because a term has only one representation (apart from renaming free variables) and the "names" of bound variables are determined by the structure of the term. There is also a downside associated with this, namely that changing the structure of a term (e.g. by doing a β-reduction) also requires changing the indices of the bound variables. This is not as big of an issue because Grobauer [1] uses de Bruijn indices and includes a host of lemmas that deal with the index shifting. Furthermore, there is a library included in the Isabelle/HOL distribution which implements the lambda calculus using de Bruijn indices, including lemmas about β- and η-reduction.

In the end, the decision was made in favor of de Bruijn indices, mainly because it allows us to use Grobauer's thesis [1] and the existing Isabelle library of the lambda calculus as a basis. However, because this decision had to be made in advance, it is not clear whether using de Bruijn indices really was easier than if we would have used named bound variables.

4.2 Formalizing the Lambda Calculus

The existing Isabelle implementation of the lambda calculus provided a very solid basis for our formalization. The only adaptation that had to be made was to extend the datatype to enable the distinction between free and bound variables, as discussed in Section 2.1. We went from

```
datatype dB =
          Var nat
        | App dB dB (infixl "°" 200)
        | Abs dB
```

to

```
datatype dB =
          BVar nat
        | Var nat
        | App dB dB (infixl "°" 200)
        | Abs dB
```

where `BVar` represents bound variables, `Var` free variables, `App` function application and `Abs` abstraction.

As a result, the definitions and proofs that used a case distinction or induction on the structure of the datatype had to be extended to include the new `BVar` case. Because in most definitions and proofs the case for free variables is trivial and the case for variables (`Var` in the old datatype) can be reused as the case for bound variables (`BVar` in the new datatype), we first replaced all instances of `Var` by `BVar`. We then added the additional case for free variables (`Var` in the new datatype). The proofs for the `BVar` cases needed no modification. To prove the `Var` cases, it was usually enough to use Isabelle's automated proof methods (if the case is trivial) or to copy the proof from the `BVar` case. For some proofs it was necessary to duplicate helper lemmas that were defined for `BVar`. An example of this is the following lemma which is part of the theory on strong normalization and which states that a `Var` does not have infinite chains of β-reductions (`IT` is an inductive characterization of all terms satisfying this property).

```
lemma Var_IT: "IT (Var n)"
```

The proof for this lemma can be copied from the corresponding lemma `BVar_IT`. These lemmas are later used to show that simply typed terms have no infinite chains of β-reductions.

Fortunately, it was never necessary to devise a completely new proof.

4.2.1 Lemmas from Grobauer's Thesis

After adapting the parts of the lambda calculus implementation that we needed, we proceeded by formalizing the lemmas and facts from the appendix of Grobauer's thesis ([1] p. 52 ff.). These lemmas state properties of the lambda calculus (including some properties of de Bruijn indices) and are used in the termination and correctness proofs of the unification algorithm. Most of the results could be formalized in a more or less straightforward way by translating the written proofs to Isabelle, although some additional definitions were needed. For example, to express the following lemma, we had to define a relation `beta_m` to express m successive β-reductions and an accompanying function `list_subst` which successively substitutes a list of variables and replaces the ellipsis in the following lemma.

Lemma 15

$$(\lambda_m\ t)\ (\!|\overline{x_m}|\!) \xrightarrow{m}_\beta t\,[x_{m-1} + (m-1)\,/\,m-1]\ldots[x_0 + 0/0]$$

```
inductive beta_m :: "nat ⇒ dB ⇒ dB ⇒ bool" where
"beta_m 0 t t" |
"beta r s ⟹ beta_m m s t ⟹ beta_m (Suc m) r t"

fun list_subst :: "dB ⇒ nat list ⇒ nat ⇒ dB" where
"list_subst t [] _ = t" |
"list_subst t (x # xs) m =
list_subst (t[BVar (x + m - 1) / m - 1]) xs (m - 1)"
```

To express the term on the right hand side of this lemma with *list_subst*, we would write `list_subst t x_m m`, where `x_m` contains the indices of the variables in $\overline{x_m}$.

The proofs of Lemma 22 and Fact 13 (see below) of Grobauer's thesis [1] required some more work and also some additional assumptions. For Lemma 22 this is described in Chapter 3. The extensions required to prove Fact 13 are described in the following.

Fact 13 *Let s and t have no loose bound variables and let $s(\!|\overline{x_m}|\!) \equiv t(\!|\overline{y_n}|\!)$. If for $\overline{z'_m}$ and $\overline{z_n}$ it is $z'_i = z_j$ whenever $x_i = y_j$ then $s(\!|\overline{z'_m}|\!) \equiv t(\!|\overline{z_n}|\!)$.*

Fact 13 is one of four similar facts, the other three being Facts 10 through 12 (Facts 10 and 12 were already mentioned in Chapter 3). The proofs are also supposed to work in a similar way, in particular, we apply induction on the length of one of the argument lists and then use either Lemma 22 or Lemma 23 (see below) to show the induction step.

Lemma 23 *Let s and t have no loose bound variables and let*

$$t(\!|xs +\!\!+ x{:}\overline{x_m}|\!) \equiv s(\!|\overline{y_n}|\!)$$

for two lists of bound variables $xs +\!\!+ x{:}\overline{x_m}$ *and* $\overline{y_n}$*. Then*

- *if* $x \in \overline{y_n}$ *and* $x \notin \{xs\}$ *it is* $t(\!|xs +\!\!+ z{:}\overline{x'_m}|\!) \equiv s(\!|\overline{y'_n}|\!)$ *for any bound variable* z *where*

$$x'_i = \begin{cases} x_i & \text{if } x_i \neq x \\ z & \text{otherwise} \end{cases}$$

and

$$y'_i = \begin{cases} y_i & \text{if } y_i \neq x \\ z & \text{otherwise} \end{cases}$$

for any bound variable z

- *if* $x \notin \overline{y_n}$ *there exists a* t' *such that* $t \xrightarrow{*}_{\beta\eta} \lambda_{|xs|+1}\ t'$ *and* $\neg\,(\mathit{free}\ t'\ 0)$

For Facts 10 through 12 the proofs work as described. The only change we had to make was to strengthen the statements of Lemmas 22 and 23. The proofs described by Grobauer already prove these stronger versions (assuming the input terms to be simply typed for Lemma 22, cf. Chapter 3).

If we try to prove Fact 13 in the same way however, we discover that we are not able to prove the assumptions of the induction hypothesis. We tried to generalize the statement of Fact 13 in multiple ways, but the induction hypothesis always had at least one assumption that we could not prove. We then had the idea of dividing the proof into two steps. The first step would be to prove a lemma of the form

$$s(\!|zs +\!\!+ \overline{x_m}|\!) \equiv t(\!|\overline{y_n}|\!) \implies s(\!|zs +\!\!+ \overline{z'_m}|\!) \equiv t(\!|\overline{z_n}|\!) \tag{4.1}$$

with some additional assumptions (see the list below for the set of assumptions we ended up with). In a second step, we would instantiate this lemma with $zs = []$. We tried to prove lemma 4.1 for the first step with different sets of assumptions, but we encountered the same issue as before. We were not able to show the assumptions of the induction hypothesis.

One of my advisors, Joshua Schneider, then came up with the following set of assumptions:

1. s and t have no loose bound variables.
2. $s(\!|zs +\!\!+ \overline{x_m}|\!) \equiv t(\!|\overline{y_n}|\!)$
3. If $x_i = y_j$ then $z'_i = z_j$.
4. The elements of $zs +\!\!+ \overline{x_m}$ are distinct. The same holds for $\overline{y_n}$ and $zs +\!\!+ \overline{z'_m}$.
5. $\overline{z'_m}$ and $\overline{y_n}$ have no common elements.
6. The de Bruijn indices of the variables in $\overline{x_m}$ are all smaller than b. The de Bruijn indices of the variables in zs, $\overline{z'_m}$ and $\overline{z_n}$ are all equal to or greater than b.

7. If the de Bruijn index of y_i is equal to or greater than b then $y_i = z_i$.

for some natural number b that we can choose. We will first show how a lemma of the form 4.1 will help us prove Fact 13 and then try to provide some intuition on why these assumptions help to prove the lemma.

It is obvious that we get the statement of Fact 13 from 4.1 if we instantiate zs with $[]$, but we also need to able to show all of the assumptions. For $zs = []$, assumptions 1 through 3 are the original assumptions of Fact 13, so we don't need to worry about them. In order to fulfill assumption 4, we have to assume that the elements of $\overline{x_m}$, the elements of $\overline{y_n}$ and the elements of $\overline{z'_m}$ are distinct. This turns out not to be a problem because the only place in Grobauer's thesis where Fact 13 is used, these assumptions are fulfilled. To fulfill assumption 5 through 7, we can do the following. We increase the indices of the variables in $\overline{z'_m}$ and $\overline{z_n}$ by one plus the highest index of a variable in $\overline{x_m}$ and $\overline{y_n}$. It is easy to see that this fulfills the assumptions, the problem we now have is that if we use lemma 4.1 with the modified $\overline{z'_m}$ and $\overline{z_n}$, its statement will also only hold for the modified lists. This turns out not to be a problem, we can simply decrease the indices of the variables in the lists by the same amount again and it will not affect $\beta\eta$-equality. This can be shown by using the assumption that s and t contain no loose bound variables (Grobauer mentions that such a lemma is needed for proving Fact 13). Using this, we are able to show Fact 13 with the additional assumption that the elements of $\overline{x_m}$, the elements of $\overline{y_n}$ and the elements of $\overline{z'_m}$ are distinct.

The reason why we need the additional list zs is because, in the induction step, we want to somehow remove an element of $\overline{x_m}$ so that we can apply the induction hypothesis. A first idea could be to integrate the first element of $\overline{x_m}$ into s, i.e. instead of $s(\!|\overline{x_m}|\!)$, we would write $(s \mathbin{@} x_{m-1})(\!|[x_{m-2}, \ldots, x_0]|\!)$. The problem with this approach is that $s \mathbin{@} x_{m-1}$ now contains a loose bound variable which contradicts one of the assumptions. Having a second list allows us to move the first element of $\overline{x_m}$ into zs, i.e. we go from $zs +\!\!+ \overline{x_m}$ to $(zs +\!\!+ [x_{m-1}]) +\!\!+ [x_{m-2}, \ldots, x_0]$.

Understanding why assumptions 4 through 7 are necessary is easiest when trying to construct a formal proof without them, but the intuition is the following. When we apply Lemma 23 in the induction step, we either end up in the first case which tells us that we can replace x by any z we choose or we end up in the second case which tells us that x will disappear after a number of β- and η-transitions, effectively also telling us that we can replace it by anything. Our strategy for the proof is to replace x by the corresponding variable in $\overline{z'_m}$ in order to eventually end up at the desired statement $s(\!|zs +\!\!+ \overline{z'_m}|\!) \equiv t(\!|\overline{z_n}|\!)$. To make this work, we need to ensure that every variable in $\overline{x_m}$ is replaced exactly once, otherwise we might run into a situation where we first replace the variable x by z and in a later step we replace x' by z'. If we

have $x' = z$, we will end up replacing x by the wrong variable. Assumptions 4 through 7 prevent this by ensuring that all variables that have already been replaced have indices equal to or greater than b and that all variables that have not yet been replaced have indices smaller than b. In our previous example, this would ensure that $x' \neq z$ and prevent the variable x' from being replaced a second time.

Unfortunately, due to time constraints, we were not able to complete the formal proof of Fact 13. The proof attempt in the Isabelle theory contains some unproved statements, however, we are quite sure that it is possible to prove the lemma using these assumptions.

4.2.2 General Facts about the Lambda Calculus

In order to formally prove the lemmas from Grobauer's thesis [1] we had to formalize some general facts about the lambda calculus. Two of the most important ones are the Church-Rosser property of $\beta \cup \eta$ and the fact that simply typed lambda terms have no infinite chains of β- and η-reductions.

A proof of the Church-Rosser property is included in the existing Isabelle implementation of the lambda calculus, so all we had to do was extend this proof to the new datatype which fortunately did not require many changes.

The existing implementation also includes a proof of the strong normalization property of simply typed terms, but only with respect to β-reduction. Because we needed $\beta\eta$-normal forms in the proof of Lemma 22, we had to extend the strong normalization property to $\beta \cup \eta$.

Showing the strong normalization property for η-reduction is simple, because the size of the term decreases in every step which makes an infinite chain of reductions impossible. However, combining the strong normalization properties for chains of β- and chains of η-reductions to chains of $\beta \cup \eta$ reductions is not as simple. It required proving an η-postponement theorem which states that if we have $s \xrightarrow{*}_\eta t$ and $t \longrightarrow_\beta u$, there exists a t' such that $s \xrightarrow{+}_\beta t'$ and $t' \xrightarrow{*}_\eta u$, i.e. we can move the η-reductions to the end of the chain (we might have to add or remove some reductions). The existing lambda calculus implementation already contains an eta postponement theorem, however, it is a weaker version, stating that if we have $s \xrightarrow{*}_\eta t$ and $t \xrightarrow{*}_\beta u$, there exists a t' such that $s \xrightarrow{*}_\beta t'$ and $t' \xrightarrow{*}_\eta u$. In this version of the theorem, the number of β-reductions can decrease. This turns out to be a problem for the proof of strong normalization. This part of the formal proof was done by one of my advisors, Dmitriy Traytel. The exact details can be found in the Isabelle theory.

Chapter 5

The Unification Algorithm and Formal Proof of Correctness

Nipkow [5] presents his unification algorithm first as a set of rules and then gradually transforms it into a functional program. The version shown here is the first version of the algorithm Grobauer's thesis ([1] p.9). It is closer to a functional program than Nipkow's first version, but there are some simplifications. In addition to the concepts and notation introduced in Chapter 2, we use $t{\downarrow_\beta}$ to denote the β-normal form of t (we assume simply typed terms in this chapter) and we use the *maybe* datatype:

datatype maybe α = None | $\lfloor\alpha\rfloor$

The *cases* function performs one step of the unification procedure. It returns either *None* if the inputs are not unifiable or a pair of values where the first value is a list of unification problems that still need to be solved in order to unify the inputs and the second value is the first part of the substitution that will have to applied to unify the inputs (cf. the definition of *unif* below). If none of the given patterns matches the input, the function returns *None*.

$$\text{cases } \left(x(\!|\overline{s_n}|\!) \stackrel{?}{=} x(\!|\overline{t_n}|\!)\right) = \llcorner\left\langle\left[s_{n-1} \stackrel{?}{=} t_{n-1}, \ldots, s_0 \stackrel{?}{=} t_0\right], \emptyset\right\rangle\lrcorner \qquad \text{(rigid-rigid)}$$

$$\text{cases } \left(F(\!|\overline{x_m}|\!) \stackrel{?}{=} y(\!|\overline{t_n}|\!)\right) = \qquad \text{(flex-rigid)}$$

$$\textbf{if } F \in \mathcal{FV}\left(\overline{t_n}\right) \vee y \notin \{\overline{x_m}\} \textbf{ then None else}$$

$$\llcorner\left\langle\left[H_{n-1}(\!|\overline{x_m}|\!) \stackrel{?}{=} t_{n-1}, \ldots, H_0(\!|\overline{x_m}|\!) \stackrel{?}{=} s_0\right], \left\{F \mapsto \lambda_m \ \left(\underline{j}\right) \ (\!|\overline{H_n(\!|zs|\!)}|\!)\right\}\right\rangle\lrcorner$$

$$\textbf{where } zs = [\underline{m-1}, \ldots, \underline{0}], x_j = y$$

$$\text{cases } \left(F(\!|\overline{x_m}|\!) \stackrel{?}{=} F(\!|\overline{y_m}|\!)\right) = \llcorner\langle[], \{F \mapsto \lambda_m \ H(\!|zs|\!)\}\rangle\lrcorner \qquad \text{(flex-flex-1)}$$

$$\textbf{where } zs = [\underline{k} \mid 0 \leq k \leq (m-1), x_k = y_k]$$

$$\text{cases } \left(F(\!|\overline{x_m}|\!) \stackrel{?}{=} G(\!|\overline{y_n}|\!)\right) = \qquad \text{(flex-flex-2)}$$

$$\llcorner\langle[], \{F \mapsto \lambda_m \ H(\!|xs|\!)\} \circ \{G \mapsto \lambda_n \ H(\!|ys|\!)\}\rangle\lrcorner$$

$$\textbf{where } |xs| = |\{xs\}| = |ys| = |\{ys\}| = |\{\overline{x_m}\} \cap \{\overline{y_n}\}|,$$

$$x_{xs_i} = y_{ys_i} \text{ for } 0 \leq i \leq (|xs| - 1)$$

Before passing a unification problem to *cases*, we first remove leading lambdas using on-the-fly η-transformation and orient it in such a way that we do not need a separate *rigid-flex* case in the *cases* function. These two steps are done by the following two functions.

$$\begin{aligned}
&\text{otf}\eta \ \left(s \stackrel{?}{=} t\right) = \textbf{case } \left(s \stackrel{?}{=} t\right) \textbf{ of} \\
&\quad \mid \left(\lambda\ s' \stackrel{?}{=} \lambda\ t'\right) \to \text{otf}\eta \ \left(s' \stackrel{?}{=} t'\right) \\
&\quad \mid \left(\lambda\ s' \stackrel{?}{=} t\right) \to \text{otf}\eta \ \left(s' \stackrel{?}{=} (\text{lift } t\ 0)\ @\ \underline{0}\right) \\
&\quad \mid \left(s \stackrel{?}{=} \lambda\ t'\right) \to \text{otf}\eta \ \left((\text{lift } s\ 0)\ @\ \underline{0} \stackrel{?}{=} t'\right) \\
&\quad \mid \textbf{otherwise} \to s \stackrel{?}{=} t
\end{aligned}$$

$$\begin{aligned}
&\text{tw} \ \left(s \stackrel{?}{=} t\right) = \textbf{case } s \textbf{ of} \\
&\quad \mid F(\!|_|\!) \to \left(s \stackrel{?}{=} t\right) \\
&\quad \mid \textbf{otherwise} \to \left(t \stackrel{?}{=} s\right)
\end{aligned}$$

Finally, we define *unif* function which solves a list of unification problems.

$$\begin{array}{l}\text{unif } \langle uus, \varphi\rangle = \textbf{case } uus \textbf{ of}\\ \quad \mid [] \to \llcorner \varphi \lrcorner\\ \quad \mid u{:}us \to \textbf{case } (\text{cases} \circ \text{tw} \circ \text{otf}\eta)\, u \textbf{ of}\\ \qquad \mid \text{None} \to \text{None}\\ \qquad \mid \llcorner \langle us', \varphi'\rangle \lrcorner \to \text{unif } \langle us' +\!\!+ (\varphi' us)\downarrow_\beta, \varphi' \circ \varphi\rangle\end{array}$$

As mentioned previously, the algorithm as it is shown here is not yet a valid functional program because we have, for example, the equation $zs = [\underline{k} \mid 0 \leq k \leq (m-1), x_k = y_k]$ in the *flex-flex-1* case. In the Isabelle formalization, such an equation has to be replaced by a function calculating zs when given $\overline{x_m}$ and $\overline{y_m}$. A possible Isabelle implementation is the following:

```
fun flex_flex_1_zs :: "dB list ⇒ dB list ⇒ nat ⇒ dB list" where
"flex_flex_1_zs [] [] _ = []" |
"flex_flex_1_zs (BVar x # xs) (BVar y # ys) k = (if x = y
        then BVar k # (flex_flex_1_zs xs ys (k - 1))
        else flex_flex_1_zs xs ys (k - 1))"
```

which lets us calculate zs as `flex_flex_1_zs x_m y_m (m - 1)`. Another example where we need to modify the algorithm in order to implement it in Isabelle occurs in all cases except for the *rigid-rigid* one. In the description above, we assume that H is a new free variable that has not been used before (and doesn't occur in any of the unification problems that follow the current one in the list of unification problems that is being solved by *unif*). This can be implemented by adding another parameter to the *cases* function which represents the highest index of all free variables (remember that we use natural numbers to name free variables) occurring in the list of unification problems of *unif* and then choosing a higher value for the index of H.

In order to prove that *unif* terminates, Grobauer proposes the following ([1] p. 10 ff.). We modify the *flex-rigid* case of the *cases* function such that it adds a marker symbol $\bullet$ at the end of the list of unification problems. The *flex-rigid* case therefore now returns either *None* or

$$\llcorner \left\langle \left[H_{n-1}(\!|\overline{x_m}|\!) \stackrel{?}{=} t_{n-1}, \ldots, H_0(\!|\overline{x_m}|\!) \stackrel{?}{=} s_0, \bullet \right], \left\{ F \mapsto \lambda_m \ \left(\underline{j}\right) (\!|\overline{H_n(\!|zs|\!)}|\!) \right\} \right\rangle \lrcorner$$

If the *unif* function encounters a marker symbol, it will simply discard it and move on to the next unification problem.

Grobauer then makes the following observations about the list of unification problems that is manipulated throughout the execution of the *unif* function.

- There is never a free variable which was introduced in a *flex-rigid* case on a right hand side of a unification problem in front of the last marker.
- If there is no marker present, there is no free variable which was introduced in a *flex-rigid* case in any unification problem of the list.
- All the left hand side of unification problems in front of the last marker are of the form $H(\!|\overline{x_m}|\!)$ where H is a free variable that was introduced in a *flex-rigid* case.
- Only cases *flex-rigid* and *flex-flex-2* can occur in front of the last marker.

Grobauer then defines a termination measure on the list of unification problems and proves the termination of the *unif* function by a case distinction and by making use of the observations listed above.

If we try to implement this approach in Isabelle, we run into some issues.

The first issue is that we somehow need to keep track of the variables that were introduced in a *flex-rigid* case. We can do this by passing the set of variables added in *flex-rigid* cases to the *cases* function and then returning the updated set back to *unif*.

The second issue concerns the observations made by Grobauer. If we consider a list of unification problems that does not contain a marker symbol, it can easily be seen that the observations are correct. Moreover, if we use this list as an input to the *unif* function, we can see that the observations hold after every step (i.e. after every invocation of the *cases* function). On the other hand, it is obvious that there exists a list of unification problems that does not fulfill the observations.

If we want to show the termination of the *unif* function, we need to show that it terminates for every possible input. However, because the proof of termination relies on the observations, we won't be able to show termination for inputs not satisfying the observations.

We can solve this problem in two ways. The first option is to restrict the set of possible inputs to the *unif* function by defining a new datatype. This datatype contains a subset of all possible lists of unification problems, namely it contains only those for which the observations hold. The second option is to limit the proof of termination to a subset of all possible inputs, i.e. we add an assumption to the termination proof which ensures that the inputs satisfy the observations.

Unfortunately, due to time constraints, we did not get very far with the proof of termination. The Isabelle theory includes the definitions of the functions discussed above, with the necessary changes made to accommodate the marker symbol and to keep track of the variables introduced in *flex-rigid* cases. Moreover, the theory includes a definition of the termination metric proposed by Grobauer.

Chapter 6

Conclusion

We formally proved most of the auxiliary lemmas Grobauer [1] uses to show termination and correctness of the unification algorithm. We also discovered that formally proving some of the lemmas requires some more care than what is described in Grobauer's thesis. In particular, we showed that Lemma 22 and Facts 10 and 12 are not true for untyped lambda terms. We also showed that this extends to Nipkow's [5] unification algorithm which does not always return the most general unifier for untyped terms. Furthermore, we implemented the unification algorithm in Isabelle and made a first effort towards proving its termination. We also discussed how the termination proof from Grobauer has to be adapted. Finally, we discussed advantages disadvantages of using de Bruijn indices for formal proofs.

Unfortunately, we were not able to formally prove the correctness of the unification algorithm. Completing the proof of correctness would be the obvious continuation of the work of this thesis. Moreover, one could try to formalize the enhanced versions of the unification algorithm which are detailed by Nipkow and contain efficiency improvements. Formalizing these extensions could be particularly interesting if the Isabelle specification was made executable. Then one could have an efficient, executable and verified unification algorithm for higher-order pattern unification.

Bibliography

[1] Bernd Grobauer. A verified unification algorithm for higher-order patterns. Diploma thesis, Technische Universität München, 1997.

[2] Gerard P. Huet. The undecidability of unification in third order logic. *Information and Control*, 22(3):257–267, 1973.

[3] Gerard P. Huet. A unification algorithm for typed λ-calculus. *Theoretical Computer Science*, 1(1):27 – 57, 1975.

[4] Dale Miller. A logic programming language with lambda-abstraction, function variables, and simple unification. In Peter Schroeder-Heister, editor, *Extensions of Logic Programming*, pages 253–281, Berlin, Heidelberg, 1991. Springer Berlin Heidelberg.

[5] Tobias Nipkow. Functional unification of higher-order patterns. In *Proc. 8th IEEE Symp. Logic in Computer Science*, pages 64–74, 1993.

ETH

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Declaration of originality

The signed declaration of originality is a component of every semester paper, Bachelor's thesis, Master's thesis and any other degree paper undertaken during the course of studies, including the respective electronic versions.

Lecturers may also require a declaration of originality for other written papers compiled for their courses.

I hereby confirm that I am the sole author of the written work here enclosed and that I have compiled it in my own words. Parts excepted are corrections of form and content by the supervisor.

Title of work (in block letters):

Towards Formalizing Higher-Order Pattern Unification in Isabelle/HOL

Authored by (in block letters):

For papers written by groups the names of all authors are required.

Name(s):	**First name(s):**
Angthan	Roman

With my signature I confirm that

- I have committed none of the forms of plagiarism described in the 'Citation etiquette' information sheet.
- I have documented all methods, data and processes truthfully.
- I have not manipulated any data.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for plagiarism.

Place, date	**Signature(s)**
Schipfheim, 3.9.2020	

For papers written by groups the names of all authors are required. Their signatures collectively guarantee the entire content of the written paper.